MODULE M203 · ATELIER 3

Gestion d’un centre de formation

Procédures & fonctions stockées MySQL — Rapport illustré

Basma Elmaimouni

Atelier 3 – M203

MySQL 8.4 sur Aiven · Terminal Windows

12 novembre 2026

```
CALL valider_inscription(1);
```

Contexte et objectifs

TechLearn : automatiser la gestion des sessions de formation

Contexte

Le centre **TechLearn** propose des sessions animées par des formateurs. Chaque session a un formateur, une **capacité** et un nombre d'**inscrits**.

L'administration veut automatiser : taux d'occupation, mise à jour des inscrits, suppression des sessions vides, rapports.

Base de données

- **formateurs** (id, nom, specialite)
- **stagiaires** (id, nom, ville)
- **sessions** (id, intitule, formateur_id, capacite, inscrits)
- Clé étrangère sessions → formateurs

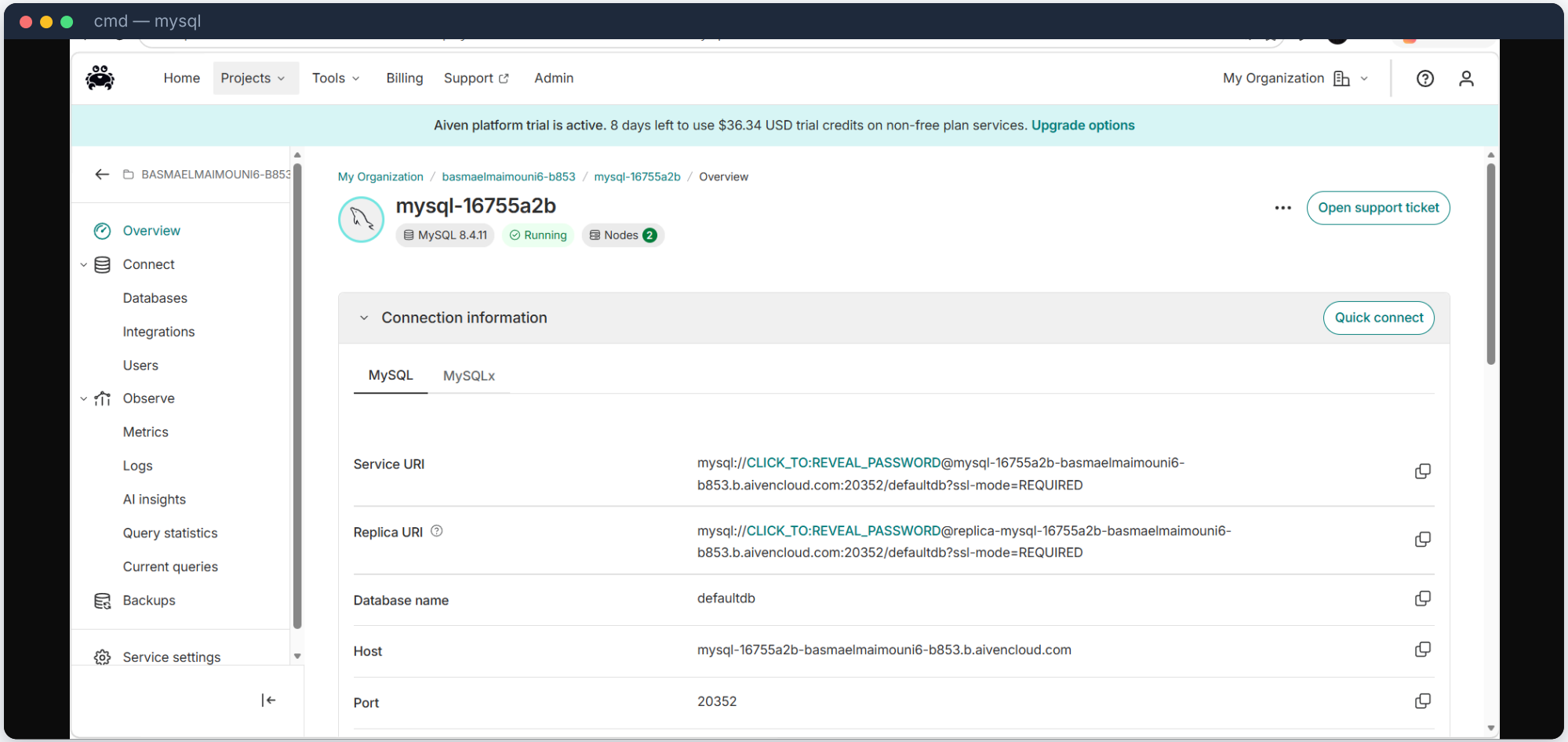
Objectifs pédagogiques

- Créer procédures et fonctions stockées
- Maîtriser les paramètres IN / OUT / INOUT
- Utiliser IF / ELSE et WHILE
- Documenter un travail technique

Étape	Contenu du rapport (captures d'écran)
01 – 02	Service Aiven, connexion terminal, création de la base
03 – 05	Création des tables, vérification, insertion des données
06 – 08	Exercices 1 à 5 : IN, OUT, INOUT, fonction, boucle WHILE
09 – 11	Défis finaux : sessions_formateur, supprimer_sessions_vides, valider_inscription
12 – 13	Analyse des résultats et conclusion personnelle

01 Service MySQL sur Aiven

Un serveur MySQL 8.4.11 dans le cloud



Le service **mysql-16755a2b** est au statut **Running** (2 nœuds). La section *Connection information* fournit l'hôte, le port **20352**, la base **defaultdb** et l'utilisateur **avnadmin**.

Statut : Running | MySQL 8.4.11 | Connexion SSL

02 Connexion et création de la base

Terminal Windows → client mysql 8.4.9

cmd — mysql

C:\Windows\System32\cmd.e

```
Microsoft Windows [version 10.0.26200.9711]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\lenovo\Desktop\atelier 3_203>"C:\Program Files\MySQL\MySQL Server 8.4\bin\mysql.exe" -h mysql-16755a2b-basmaelmaimouni6-b853.b.aivencloud.com -P 20
352 -u avnadmin -p --ssl-mode=REQUIRED --ssl-ca=ca.pem defaultdb
WARNING: no verification of server certificate will be done. Use --ssl-mode=VERIFY_CA or VERIFY_IDENTITY.
Enter password: ************************
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 1690
Server version: 8.4.11 Source distribution

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE techlearn_db;
Query OK, 1 row affected (4.36 sec)

mysql> USE techlearn_db;
Database changed
mysql> USE defaultdb;
Database changed
mysql> SELECT DATABASE();
+------------+
| DATABASE() |
+------------+
| defaultdb  |
+------------+
1 row in set (0.79 sec)

mysql> SHOW TABLES;
Empty set (0.16 sec)

mysql> INSERT INTO formateurs (nom, specialite) VALUES
    -> ('Mme Nadia', 'Bases de données'),
    -> ('M. Karim', 'Développement Web'),
    -> ('Mme Salma', 'Analyse de données');
```

Connexion avec `--ssl-mode=REQUIRED --ssl-ca=ca.pem`. Le mot de passe est masqué (étoiles). L'invite **mysql>** confirme la connexion, puis `CREATE DATABASE techlearn_db` réussit. Les objets de l'atelier sont créés dans **defaultdb** (`SELECT DATABASE()`).

Welcome to the MySQL monitor | Query OK, 1 row affected | Base courante : defaultdb

03 Création des tables (1/2)

formateurs et stagiaires

cmd — mysql

```
mysql> CREATE TABLE formateurs (
    ->   id INT AUTO_INCREMENT PRIMARY KEY,
    ->   nom VARCHAR(100),
    ->   specialite VARCHAR(100)
    -> );
Query OK, 0 rows affected (1.00 sec)

mysql>
mysql> CREATE TABLE stagiaires (
    ->   id INT AUTO_INCREMENT PRIMARY KEY,
    ->   nom VARCHAR(100),
    ->   ville VARCHAR(100)
    -> );
Query OK, 0 rows affected (0.28 sec)
```

Création des tables `formateurs` et `stagiaires` avec clé primaire auto-incrémentée. Les *Query OK* prouvent la bonne exécution.

2 tables créées | PRIMARY KEY AUTO_INCREMENT

04 Création des tables (2/2)

sessions + clé étrangère, puis vérification

cmd — mysql

C:\Windows\System32\cmd.e

```
    ->   ville VARCHAR(100)
    -> );
Query OK, 0 rows affected (0.28 sec)

mysql>
mysql> CREATE TABLE sessions (
    ->   id INT AUTO_INCREMENT PRIMARY KEY,
    ->   intitule VARCHAR(100),
    ->   formateur_id INT,
    ->   capacite INT,
    ->   inscrits INT DEFAULT 0,
    ->   FOREIGN KEY (formateur_id) REFERENCES formateurs(id)
    -> );
Query OK, 0 rows affected (0.15 sec)

mysql> SHOW TABLES;
+---------------------+
| Tables_in_defaultdb |
+---------------------+
| formateurs          |
| sessions            |
| stagiaires          |
+---------------------+
3 rows in set (0.17 sec)

mysql> INSERT INTO formateurs (nom, specialite) VALUES
    -> ('Mme Nadia', 'Bases de données'),
    -> ('M. Karim', 'Développement Web'),
    -> ('Mme Salma', 'Analyse de données');
Query OK, 3 rows affected (0.19 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO stagiaires (nom, ville) VALUES
    -> ('Ali', 'Casablanca'),
    -> ('Sara', 'Rabat'),
    -> ('Mouna', 'Marrakech');
Query OK, 3 rows affected (0.15 sec)
Records: 3  Duplicates: 0  Warnings: 0

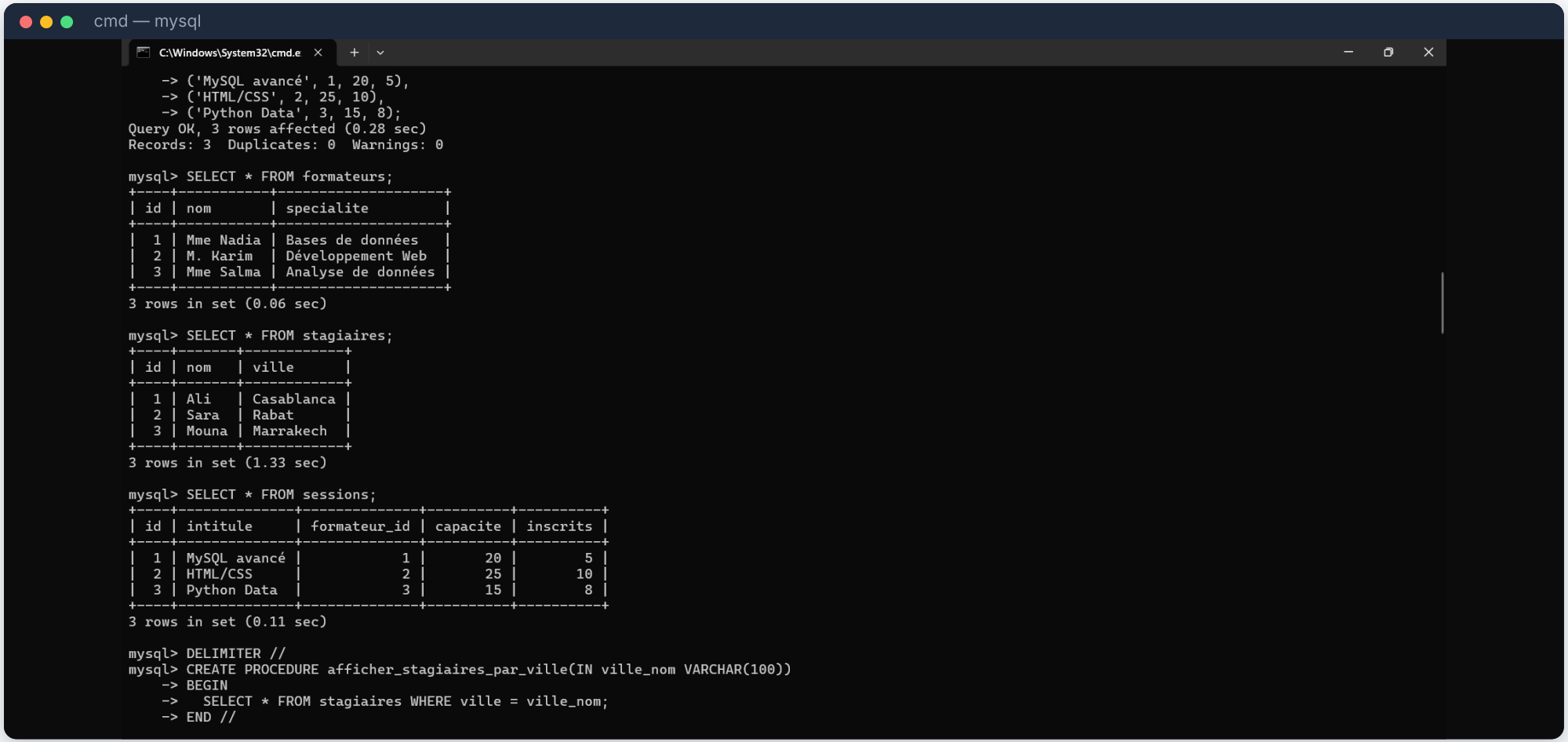
mysql> INSERT INTO sessions (intitule, formateur_id, capacite, inscrits) VALUES
    -> ('MySQL avancé', 1, 20, 5),
```

La table `sessions` référence `formateurs(id)` par une clé étrangère. `SHOW TABLES` affiche bien **3 tables** : formateurs, sessions, stagiaires. Les insertions commencent juste après.

FOREIGN KEY | SHOW TABLES = 3 lignes | 3 formateurs insérés

05 Insertion des données

Jeu de test du centre TechLearn




```
    -> ('MySQL avancé', 1, 20, 5),
    -> ('HTML/CSS', 2, 25, 10),
    -> ('Python Data', 3, 15, 8);
Query OK, 3 rows affected (0.28 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM formateurs;
+----+-----------+--------------------+
| id | nom       | specialite         |
+----+-----------+--------------------+
|  1 | Mme Nadia | Bases de données   |
|  2 | M. Karim  | Développement Web  |
|  3 | Mme Salma | Analyse de données |
+----+-----------+--------------------+
3 rows in set (0.06 sec)

mysql> SELECT * FROM stagiaires;
+----+-------+------------+
| id | nom   | ville      |
+----+-------+------------+
|  1 | Ali   | Casablanca |
|  2 | Sara  | Rabat      |
|  3 | Mouna | Marrakech  |
+----+-------+------------+
3 rows in set (1.33 sec)

mysql> SELECT * FROM sessions;
+----+--------------+--------------+----------+----------+
| id | intitule     | formateur_id | capacite | inscrits |
+----+--------------+--------------+----------+----------+
|  1 | MySQL avancé |            1 |       20 |        5 |
|  2 | HTML/CSS     |            2 |       25 |       10 |
|  3 | Python Data  |            3 |       15 |        8 |
+----+--------------+--------------+----------+----------+
3 rows in set (0.11 sec)

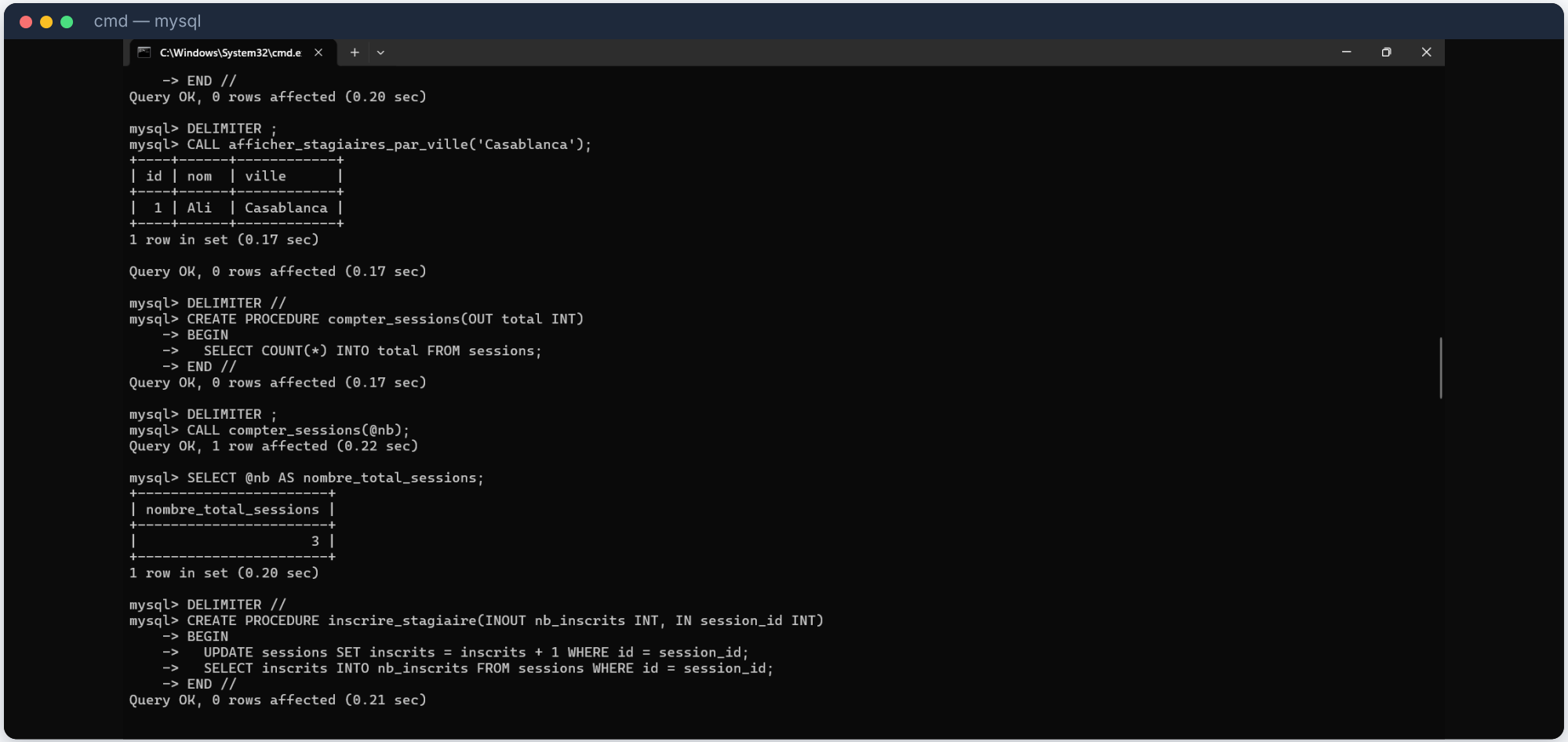
mysql> DELIMITER //
mysql> CREATE PROCEDURE afficher_stagiaires_par_ville(IN ville_nom VARCHAR(100))
    -> BEGIN
    ->   SELECT * FROM stagiaires WHERE ville = ville_nom;
    -> END //
```

Les `SELECT *` confirment : 3 formateurs, 3 stagiaires (Ali, Sara, Mouna) et 3 sessions (MySQL avancé 5/20, HTML/CSS 10/25, Python Data 8/15).

3 formateurs | 3 stagiaires | 3 sessions

06 Exercices 1 et 2 — IN et OUT

Passer une valeur / récupérer une valeur



Ex. 1 (IN) : `afficher_stagiaires_par_ville('Casablanca')` retourne **Ali**.
Ex. 2 (OUT) : `compter_sessions(@nb)` place le total dans `@nb` → **3**. La procédure de l'exercice 3 est créée en bas.

Ex.1 → Ali / Casablanca Ex.2 → 3 sessions

07 Exercices 3 et 4 — INOUT et fonction

Modifier une valeur / calculer un taux

cmd — mysql

C:\Windows\System32\cmd.e

```
mysql> DELIMITER ;
mysql> SET @nouveau = 0;
Query OK, 0 rows affected (0.14 sec)

mysql> CALL inscrire_stagiaire(@nouveau, 1);
Query OK, 1 row affected (0.11 sec)

mysql> SELECT @nouveau AS nouveau_nombre_inscrits;
+--------------------------+
| nouveau_nombre_inscrits |
+--------------------------+
|                        6 |
+--------------------------+
1 row in set (0.10 sec)

mysql> DELIMITER //
mysql> CREATE FUNCTION taux_occupation(id_session INT)
    -> RETURNS DECIMAL(5,2)
    -> DETERMINISTIC
    -> READS SQL DATA
    -> BEGIN
    ->   DECLARE taux DECIMAL(5,2);
    ->   SELECT (inscrits / capacite) * 100 INTO taux FROM sessions WHERE id = id_session;
    ->   RETURN taux;
    -> END //
Query OK, 0 rows affected (0.18 sec)

mysql> DELIMITER ;
mysql> SELECT intitule, taux_occupation(id) AS taux FROM sessions;
+--------------+-------+
| intitule     | taux  |
+--------------+-------+
| MySQL avancé | 30.00 |
| HTML/CSS     | 40.00 |
| Python Data  | 53.33 |
+--------------+-------+
3 rows in set (0.20 sec)

mysql> DELIMITER //
mysql> CREATE PROCEDURE generer_sessions(IN formateur_INT)
```

Ex. 3 (INOUT) : `inscrire_stagiaire(@nouveau, 1)` fait passer la session 1 de 5 à **6** inscrits.
Ex. 4 : la fonction `taux_occupation` donne **30,00 %**, **40,00 %** et **53,33 %**.

Ex.3 → 6 inscrits Ex.4 → 30 / 40 / 53,33 %

08 Exercice 5 — Boucle WHILE

10 sessions générées automatiquement

cmd — mysql

C:\Windows\System32\cmd.e

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE generer_sessions(IN formateur INT)
    -> BEGIN
    ->   DECLARE i INT DEFAULT 1;
    ->   WHILE i <= 10 DO
    ->     INSERT INTO sessions (intitule, formateur_id, capacite, inscrits)
    ->     VALUES (CONCAT('Session ', i, ' - Formateur ', formateur), formateur, 20, 0);
    ->     SET i = i + 1;
    ->   END WHILE;
    -> END //
Query OK, 0 rows affected (0.29 sec)

mysql> DELIMITER ;
mysql> CALL generer_sessions(2);
Query OK, 1 row affected (0.08 sec)

mysql> SELECT * FROM sessions;
+----+--------------------------+--------------+----------+----------+
| id | intitule                 | formateur_id | capacite | inscrits |
+----+--------------------------+--------------+----------+----------+
|  1 | MySQL avancé             |            1 |       20 |        6 |
|  2 | HTML/CSS                 |            2 |       25 |       10 |
|  3 | Python Data              |            3 |       15 |        8 |
|  4 | Session 1 - Formateur 2  |            2 |       20 |        0 |
|  5 | Session 2 - Formateur 2  |            2 |       20 |        0 |
|  6 | Session 3 - Formateur 2  |            2 |       20 |        0 |
|  7 | Session 4 - Formateur 2  |            2 |       20 |        0 |
|  8 | Session 5 - Formateur 2  |            2 |       20 |        0 |
|  9 | Session 6 - Formateur 2  |            2 |       20 |        0 |
| 10 | Session 7 - Formateur 2  |            2 |       20 |        0 |
| 11 | Session 8 - Formateur 2  |            2 |       20 |        0 |
| 12 | Session 9 - Formateur 2  |            2 |       20 |        0 |
| 13 | Session 10 - Formateur 2 |            2 |       20 |        0 |
+----+--------------------------+--------------+----------+----------+
13 rows in set (0.23 sec)

mysql> DELIMITER //
mysql> CREATE FUNCTION sessions_formateur(nom_formateur VARCHAR(100))
    -> RETURNS INT
    -> DETERMINISTIC
```

`generer_sessions(2)` insère 10 sessions pour le formateur 2 (id 4 à 13). La table passe de 3 à **13 lignes**.

WHILE i ≤ 10 | 13 lignes au total

09 Défis 1 et 2

Compter par formateur, nettoyer les sessions vides

cmd — mysql

C:\Windows\System32\cmd.e

```
mysql> DELIMITER //
mysql> CREATE FUNCTION sessions_formateur(nom_formateur VARCHAR(100))
    -> RETURNS INT
    -> DETERMINISTIC
    -> READS SQL DATA
    -> BEGIN
    ->   DECLARE nb INT;
    ->   SELECT COUNT(*) INTO nb
    ->   FROM sessions s
    ->   JOIN formateurs f ON s.formateur_id = f.id
    ->   WHERE f.nom = nom_formateur;
    ->   RETURN nb;
    -> END //
Query OK, 0 rows affected (0.34 sec)

mysql> DELIMITER ;
mysql> SELECT sessions_formateur('M. Karim') AS nb_sessions;
+-------------+
| nb_sessions |
+-------------+
|          11 |
+-------------+
1 row in set (0.35 sec)

mysql> DELIMITER //
mysql> CREATE PROCEDURE supprimer_sessions_vides()
    -> BEGIN
    ->   DELETE FROM sessions WHERE inscrits = 0;
    -> END //
Query OK, 0 rows affected (0.20 sec)

mysql> DELIMITER ;
mysql> CALL supprimer_sessions_vides();
Query OK, 10 rows affected (0.10 sec)

mysql> SELECT * FROM sessions;
+----+--------------+--------------+----------+----------+
| id | intitule     | formateur_id | capacite | inscrits |
+----+--------------+--------------+----------+----------+
|  1 | MySQL avancé |            1 |       20 |        6 |
|  2 | HTML/CSS     |            2 |       25 |       10 |
```

Défi 1 : `sessions_formateur('M. Karim')` retourne **11** (1 + 10 générées).
Défi 2 : `supprimer_sessions_vides()` supprime **10 lignes** (inscrits = 0).

Défi 1 → 11 | Défi 2 → 10 lignes supprimées

10 Défi 3 — valider_inscription (1/2)

IF / ELSE : cas « Inscription validée »

cmd — mysql

C:\Windows\System32\cmd.e

```
Query OK, 10 rows affected (0.10 sec)

mysql> SELECT * FROM sessions;
+----+--------------+--------------+----------+----------+
| id | intitule     | formateur_id | capacite | inscrits |
+----+--------------+--------------+----------+----------+
|  1 | MySQL avancé |            1 |       20 |        6 |
|  2 | HTML/CSS     |            2 |       25 |       10 |
|  3 | Python Data  |            3 |       15 |        8 |
+----+--------------+--------------+----------+----------+
3 rows in set (0.38 sec)

mysql> DELIMITER //
mysql> CREATE PROCEDURE valider_inscription(IN session_id INT)
    -> BEGIN
    ->   DECLARE nb_inscrits INT;
    ->   DECLARE cap INT;
    ->
    ->   SELECT inscrits, capacite INTO nb_inscrits, cap
    ->   FROM sessions WHERE id = session_id;
    ->
    ->   IF nb_inscrits >= cap THEN
    ->     SELECT 'Session complète' AS message;
    ->   ELSE
    ->     UPDATE sessions SET inscrits = inscrits + 1 WHERE id = session_id;
    ->     SELECT 'Inscription validée' AS message;
    ->   END IF;
    -> END //
Query OK, 0 rows affected (0.11 sec)

mysql> DELIMITER ;
mysql> CALL valider_inscription(1);
+---------------------+
| message             |
+---------------------+
| Inscription validée |
+---------------------+
1 row in set (0.11 sec)

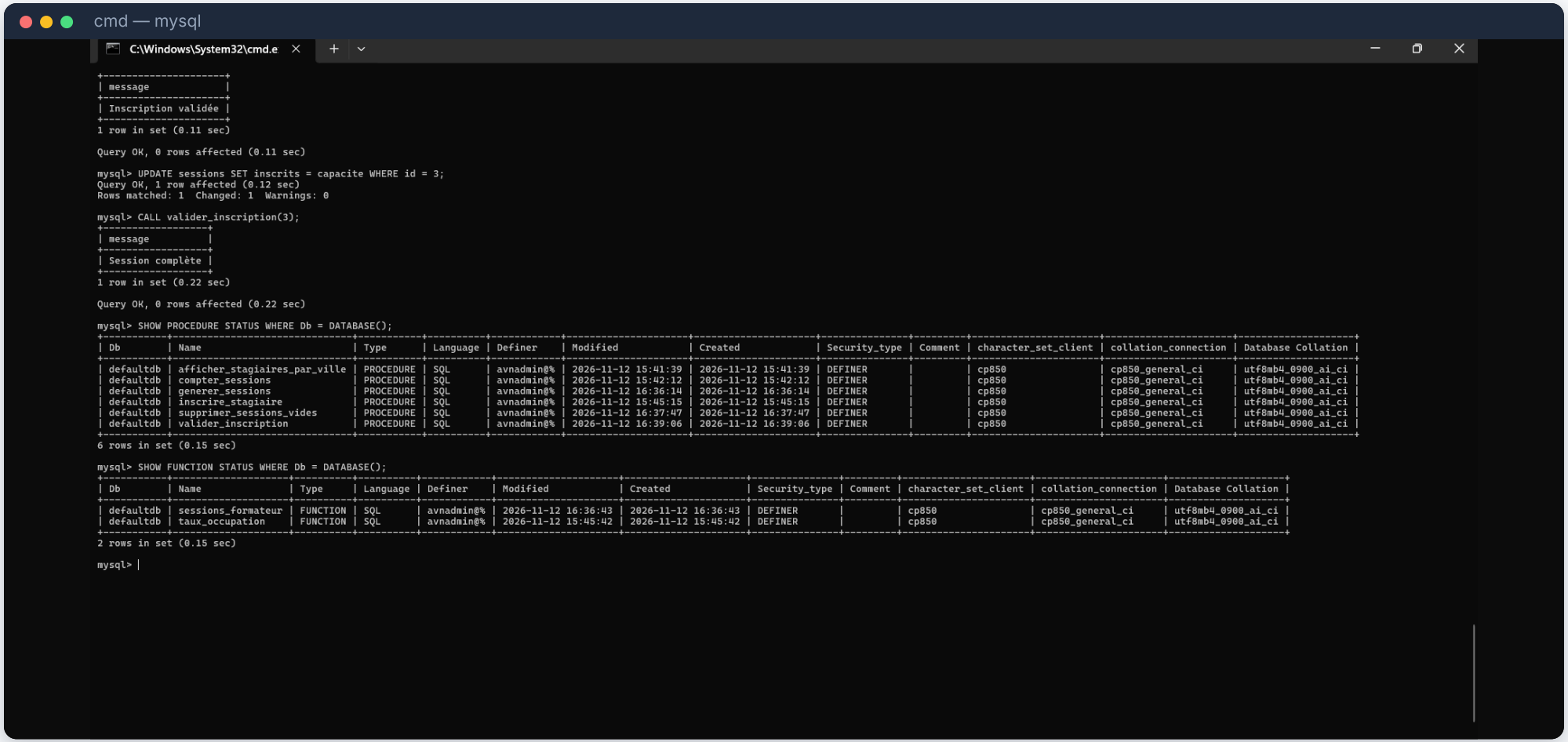
Query OK, 0 rows affected (0.11 sec)
```

Après le Défi 2, il reste **3 sessions**. `valider_inscription(1)` : la session 1 (6/20) n'est pas pleine → message **« Inscription validée »**.

3 sessions restantes | Inscription validée

11 Défi 3 — valider_inscription (2/2)

Cas « Session complète » et bilan des objets



La session 3 est rendue pleine (`inscrits = capacite`) : le message est **« Session complète »**. `SHOW PROCEDURE STATUS` / `SHOW FUNCTION STATUS` listent **6 procédures** et **2 fonctions**.

Session complète · 6 procédures · 2 fonctions

12 Analyse des résultats

Ce que fait chaque procédure ou fonction

Objet	Notion	Rôle et résultat observé
`afficher_stagiaires_par_ville`	IN	Filtre les stagiaires selon la ville reçue en paramètre → Ali pour « Casablanca ».
`compter_sessions`	OUT	Renvoie `COUNT(*)` dans `@nb` → 3.
`inscrire_stagiaire`	INOUT	Incrémente `inscrits` et remplace la valeur transmise → 5 devient 6.
`taux_occupation`	Fonction	Retourne (inscrits / capacité) × 100, utilisable dans un SELECT → 30 %, 40 %, 53,33 %.
`generer_sessions`	WHILE	10 insertions automatiques en un seul appel → 3 devient 13 lignes.
`sessions_formateur`	Fonction + JOIN	Compte les sessions d'un formateur par son nom → 11 pour M. Karim.
`supprimer_sessions_vides`	DELETE	Supprime les sessions à 0 inscrit → 10 lignes supprimées, 3 restantes.
`valider_inscription`	IF / ELSE	Refuse l'inscription si la session est pleine, sinon inscrit +1.

Procédure ou fonction ? Une procédure s'appelle avec `CALL` et peut renvoyer plusieurs valeurs (OUT/INOUT) ; une fonction renvoie **une seule valeur** avec `RETURN` et s'utilise dans une expression SQL.

Adaptations : `DELIMITER //` à la place de `/`, et `DETERMINISTIC READS SQL DATA` dans les fonctions (nécessaire sur Aiven).

13 Conclusion personnelle

Ce que cet atelier m’a apporté

> Cet atelier m’a fait passer de simples requêtes SQL à de véritables **blocs de logique stockés dans la base**. J’ai compris concrètement la différence entre les paramètres **IN**, **OUT** et **INOUT**, et entre procédure et fonction.

L’utilisation de **WHILE** et de **IF / ELSE** montre qu’on peut automatiser des tâches répétitives et protéger les données avec des règles métier, comme ne pas dépasser la capacité d’une session.

J’ai aussi rencontré des difficultés pratiques très formatrices : installer le client `mysql` sous Windows, me connecter à un service cloud Aiven en SSL, comprendre les erreurs (mot de passe refusé, tables inexistantes, connexion perdue ERROR 2013) et choisir un délimiteur adapté (`//`).

6	2	3	2
procédures	fonctions	types de paramètres	structures de contrôle